els do not spontaneously self-correct from injected intermediate errors (0/200 recoveries). In 98.5% of cases, they treat corrupted premises as authoritative and faithfully propagate errors downstream.

3) Finite-Sample Uncertainty Bounds: We establish that observing zero recoveries corresponds to a one-sided 95% upper bound of 1.49% under independent rollout assumptions ($n = 200$), but cluster-level modeling reveals an effective sensitivity upper bound of 13.91% ($C = 20$).
4) Evaluator Sensitivity: Heuristic regex extractors artificially penalize base models by 3% due to trailing review summaries, whereas structural equivalence demonstrates 100% baseline accuracy for both checkpoints.

II. Related Work

Self-correction and error localization in language models have attracted intense scrutiny. Table I positions RecoveryEval relative to existing paradigms.

A. Intrinsic Self-Correction Limitations

Huang et al. [7] established that prompting models to self-correct (*"Review your answer and correct mistakes"*) without external feedback impairs performance because models alter correct answers more frequently than they repair incorrect ones. Stechly et al. [10] demonstrated similar self-verification failures in planning tasks. While these studies evaluated post-hoc multi-turn prompting, RecoveryEval investigates in-flight intermediate continuation: how autoregressive models behave when continuing from corrupted intermediate derivations without explicit reflection prompts.

B. External Feedback and Process Supervision

Shinn et al. [5] (Reflexion) and Gou et al. [6] (CRITIC) demonstrated that self-correction succeeds when models interact with external tools (code interpreters, calculators, search engines). Tyen et al. [8] (BIG-Bench Mistake) showed that while models fail to locate their own mistakes, they can correct them if the exact erroneous step is pointed out. Zheng et al. [9] (ProcessBench) and Wang et al. [11] (Math-Shepherd) confirmed the difficulty of automated process verification. RecoveryEval complements these studies by providing a clean baseline for unassisted continuation, demonstrating that without external tool feedback, continuation inertia completely dominates.

III. Research Questions and Causal Framing

We formulate three core empirical questions:

- RQ1 (Continuation vs. Recovery): When presented with an unprompted continuation prefix containing an erroneous intermediate calculation, does an autoregressive language model detect and correct the error, or does it propagate the corrupted premise downstream?
- RQ2 (Protocol Sensitivity): Does the apparent interaction between model alignment and error recovery reproduce when control answer leakage, truncation censoring, and parser artifacts are eliminated?
- RQ3 (Uncertainty Quantification): When zero recovery events are observed in clustered factorial data, what exact finite-sample confidence limits are statistically defensible?

A. Causal Identification Boundaries

We represent the data-generating process as a structural causal model:

- Prefix Intervention ($T \to Y$): Within each problem cluster P_i, the problem text and remaining token horizon are held constant. The only intervention is the substitution of the Step 1 product (s_C vs. s_R). The contrast in continuation accuracy between s_C and s_R is causally identified.
- Checkpoint Comparison ($M \to Y$): Comparing Base and Instruct models is observational. Base models expect raw completion strings, while Instruct models require ChatML formatting (<|im_start|>). We do not claim to isolate a pure causal post-training effect free from interface confounding.
- Scope of Construct: The experiment measures continuation inertia under externally supplied corruption, not an intrinsic inability to self-correct during uninterrupted, self-generated reasoning.

IV. Experimental Design: Protocol 2.0

A. Benchmark Construction

We evaluate 20 multi-step arithmetic word problems inspired by GSM8K ($C = 20$). Each problem involves an initial rate-multiplication step (Step 1) followed by a downstream multiplication step to determine total revenue (Step 2). Neither prefix reveals the final terminal answer.

B. Symmetrical Counterfactual Prefix Construction

For each problem P_i, two prefixes are constructed:

- Reference Control Prefix (s_C): Contains valid Step 1 arithmetic (e.g., $5 \times 4 = 20$).
- Recovery Perturbation Prefix (s_R): Introduces an arithmetic corruption into Step 1 (e.g., $5 \times 4 = 22$).

Both prefixes are truncated immediately prior to the Step 2 calculation, ensuring exact token and character balance ($\Delta_{\text{tokens}} = 0, \Delta_{\text{chars}} = 0$).

C. Factorial Matrix and Hardware Execution

The factorial design comprises:

20 problems×2 models×2 conditions×5 seeds = 400 rollouts

Evaluated checkpoints are Qwen/Qwen2.5-Math-1.5B (Base, Rev 4a83ca6e) and Qwen/Qwen2.5-Math-1.5B-Instruct (Instruct, Rev aafeb0fc) [2]. Rollouts were executed on an NVIDIA GeForce RTX 4090 GPU in native float16 precision using stochastic nucleus sampling ($T =$